

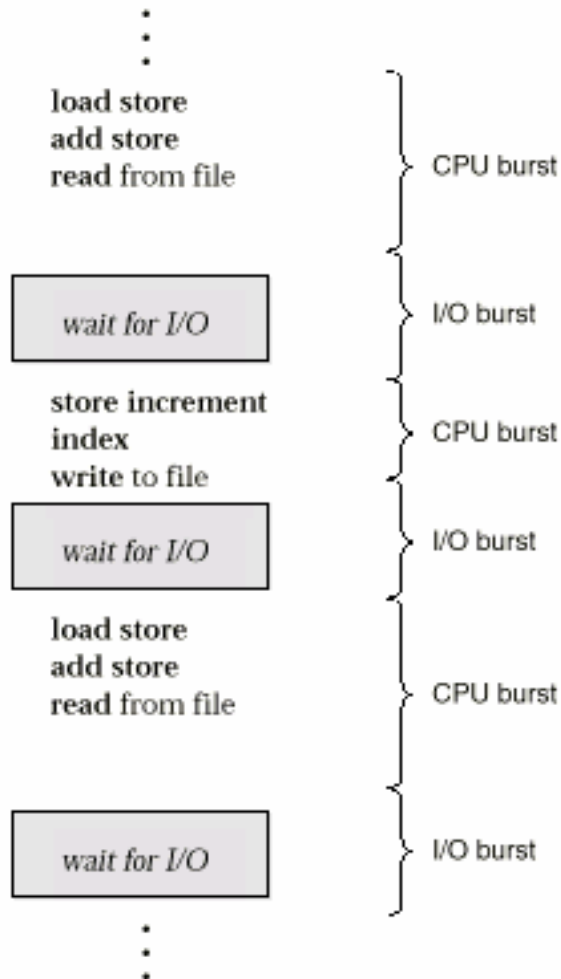
CPU스케줄링(CPU Scheduling) ~

- 프로세스 스케줄링
 - » 장기 job scheduling
 - » 단기 CPU scheduling
 - » 중기 swapping

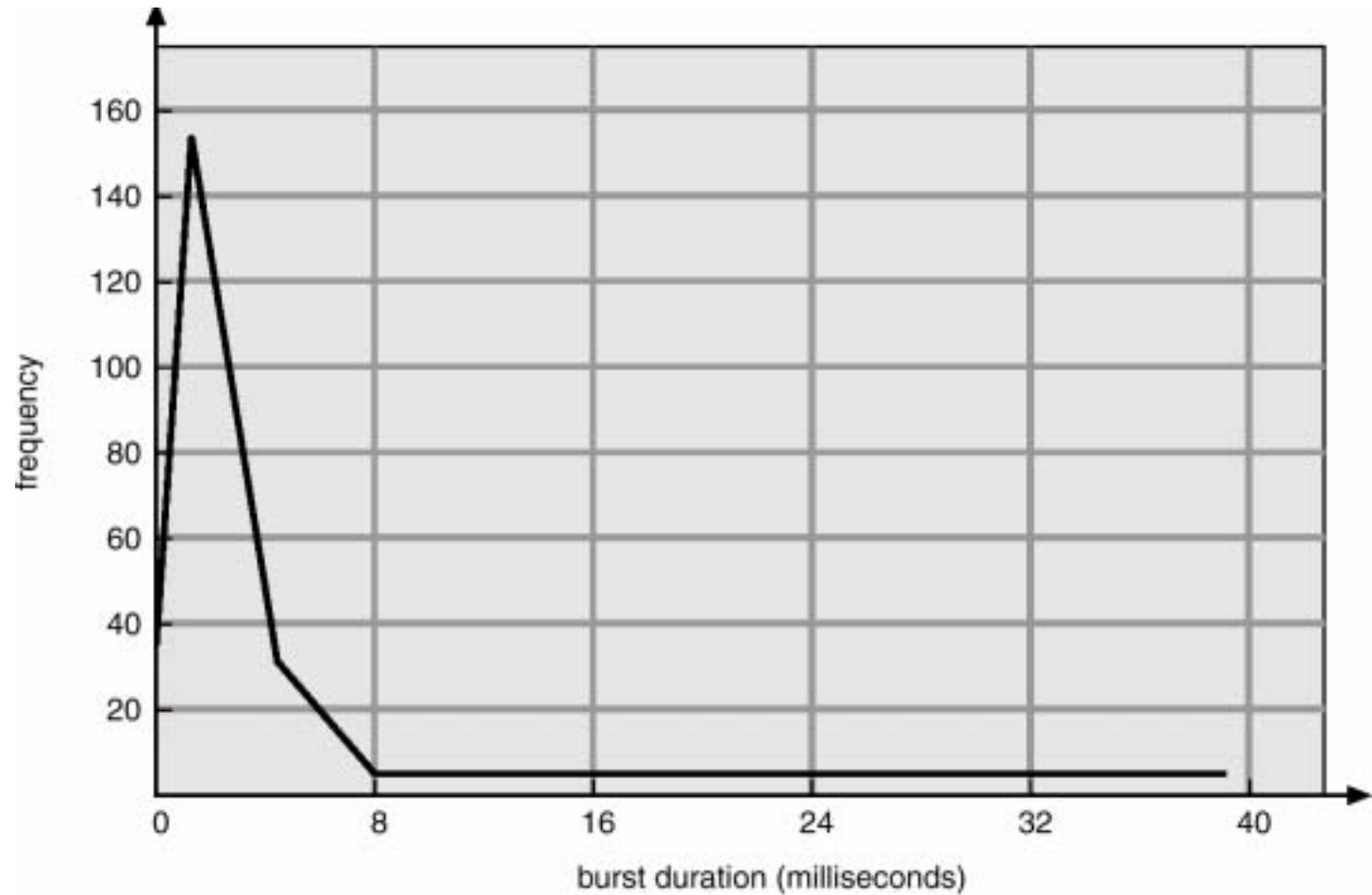
기본 개념(Basic Concepts)

- CPU-I/O 버스트 주기(burst cycle)
 - » cycle : CPU 실행(CPU burst) <- -> I/O 대기(I/O burst)
 - » CPU burst 유형
 - I/O bound program : 많은 짧은 CPU burst 가짐
 - CPU bound program : 적은 아주 긴 CPU burst 가짐
- CPU 스케줄러
 - » 단기 스케줄러(short-term scheduler) : ready queue에서 선택
 - FIFO(First-In First-Out)큐
 - 우선순위 큐
 - 트리
 - 연결리스트

Alternating Sequence of CPU And I/O Bursts



Histogram of CPU-burst Times



CPU스케줄링(CPU Scheduling) ~

- 선점 스케줄링(Preemptive Scheduling)
 - » 선점(preemptive) 스케줄링
 - 특수하드웨어(timer)필요
 - 공유 데이터에 대한 프로세스 동기화 필요
 - » 비선점(non preemptive) 또는 협조적(cooperative) 스케줄링
 - MS-Windows, 특수 하드웨어(timer) 없음
 - 종료 또는 I/O까지 계속 CPU점유
- Kernel의 선점 처리
 - » case 1(초기 Unix) : system call 완료 또는 I/O 완료할 때 까지 기다렸다가 문맥교환
 - ➔ 실시간 컴퓨팅이나 멀티 프로세싱에 나쁨
 - » case 2 : interrupt 중 다른 interrupt enable(우선순위에 따라) 선점 처리: system call만 preemptible
 - ➔ critical section(공유 데이터 수정하는 코드 부분)에 있는 동안 interrupt disable 해야 함
- CPU scheduling decision time
 1. running -> waiting : non preemptive
 2. running -> ready (interrupt) : preemptive
 3. waiting -> ready (I/O 완료) : preemptive
 4. halt : non preemptive

CPU스케줄링(CPU Scheduling)

- 분배기(Dispatcher)
 - » 문맥 교환
 - » 사용자 모드로 전환
 - » 프로그램의 적절한 위치로 점프하여 프로그램 재시작
(dispatch latency가 짧아야 함)

스케줄링 기준(Scheduling Criteria)

- 이용률(CPU utilization) : 40% ~ 90%
- 처리율(throughput) : 처리된 프로세스 개수/시간 단위
- 반환시간(turnaround time) : system in -> system out 걸린 시간
- 대기시간(waiting time) : ready queue에서 기다린 시간
- 응답시간(response time) : 대화형 시스템에서 첫 응답까지의 시간

스케줄링 알고리즘(Scheduling Algorithm)

- 척도 : 평균 대기 시간(average waiting time)
- 선입 선처리(First-Come, First-Served) 스케줄링
 - » 들어온 순서대로
 - » FIFO queue : First-in<- tail First-out<- head
 - » p141-142 예(Gantt chart)
 - » 호위 효과(convoy effect) : 큰 job하나가 끝나기를 모두 기다림(CPU-bound process가 끝나기를 I/O bounded process들이 기다림)
 - » non-preemptive임(time-sharing에서는 곤란)
- 최소 작업 우선(Shortest-Job-First) 스케줄링
 - » Shortest Next CPU Burst Scheduling
 - » 다음 CPU burst 시간이 가장 짧은 프로세스에게
 - » 두 가지 스케줄링 기법
 - non-preemptive SJF : p143 예
 - preemptive SJF = Shortest Remaining Time First Scheduling : p145 예
 - » 최적(Optimal)
 - » 단점 : 기아 상태(starvation)
 - » long-term scheduling에 좋음(프로세스 시간의 사용자 예측 치 이용)
 - » short-term scheduling 에는 나쁨 : 차기 CPU burst 시간 파악이 어려워서
 - » 차기 CPU 버스트 시간 예측 모델

First-Come, First-Served (FCFS) Scheduling

- Example:

<u>Process</u>	<u>Burst Time</u>
P_1	24
P_2	3
P_3	3

- Suppose that the processes arrive in the order: P_1 , P_2 , P_3
The Gantt Chart for the schedule is:



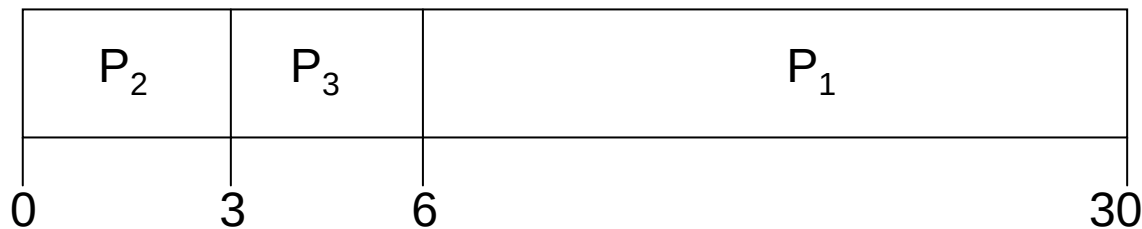
- Waiting time for $P_1 = 0$; $P_2 = 24$; $P_3 = 27$
- Average waiting time: $(0 + 24 + 27)/3 = 17$

FCFS Scheduling (Cont.)

Suppose that the processes arrive in the order

$$P_2, P_3, P_1.$$

- The Gantt chart for the schedule is:

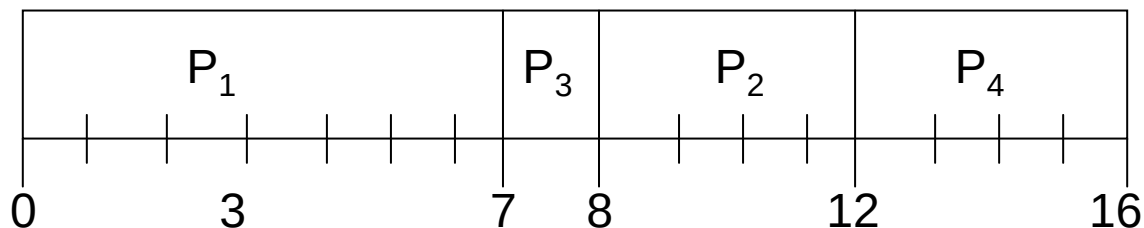


- Waiting time for $P_1 = 6$; $P_2 = 0$; $P_3 = 3$
- Average waiting time: $(6 + 0 + 3)/3 = 3$
- Much better than previous case.
- *Convoy effect* short process behind long process

Example of Non-Preemptive SJF

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

- SJF (non-preemptive)

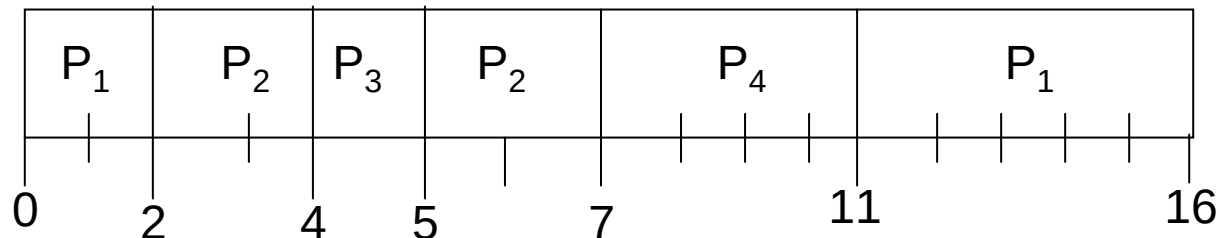


- Average waiting time = $(0 + 6 + 3 + 7)/4 = 4$

Example of Preemptive SJF

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

- SJF (preemptive)



- Average waiting time = $(9 + 1 + 0 + 2)/4 = 3$

Determining Length of Next CPU Burst

- CPU 버스트 시간 정확히 알 수는 없지만 예측 가능
- 이전 CPU 버스트 시간들의 지수적 평균(exponential averaging)으로 예측

1. t_n = actual length of n^{th} CPU burst
2. τ_{n+1} = predicted value for the next CPU burst
3. $\alpha, 0 \leq \alpha \leq 1$
4. Define :

$$\tau_{n+1} = \alpha t_n + (1 - \alpha) \tau_n.$$

Examples of Exponential Averaging

- $\alpha = 0$

- » $\tau_{n+1} = \tau_n$

- » 최근의 실제 CPU 버스트 시간은 고려 않음

- $\alpha = 1$

- » $\tau_{n+1} = t_n$

- » 마지막 실제 CPU 버스트 시간만 고려

- 식을 확장하면:

$$\begin{aligned}\tau_{n+1} &= \alpha t_n + (1 - \alpha) \tau_n \\ &= \alpha t_n + (1 - \alpha) (\alpha t_{n-1} + (1 - \alpha) \tau_{n-1}) \\ &= \alpha t_n + (1 - \alpha) \alpha t_{n-1} + (1 - \alpha)^2 \tau_{n-1} \\ &= \alpha t_n + (1 - \alpha) \alpha t_{n-1} + (1 - \alpha)^2 (\alpha t_{n-2} + (1 - \alpha) \tau_{n-2}) \\ &= \alpha t_n + (1 - \alpha) \alpha t_{n-1} + (1 - \alpha)^2 \alpha t_{n-2} + (1 - \alpha)^3 \tau_{n-3} \\ &= \alpha t_n + (1 - \alpha) \alpha t_{n-1} + (1 - \alpha)^2 \alpha t_{n-2} + (1 - \alpha)^3 \alpha t_{n-3} \dots \\ &\quad + (1 - \alpha)^j \alpha t_{n-j} + \dots \\ &\quad + (1 - \alpha)^{n+1} \tau_0\end{aligned}$$

- α 와 $(1 - \alpha)$ 이 1보다 작으므로, 후속되는 각 항목은 이전 항목보다 가중 값(weight)이 더 적어짐

스케줄링 알고리즘(Scheduling Algorithm) ~

- 우선순위(Priority) 스케줄링
 - » 높은 우선 순위의 프로세스에게
 - » ready queue = priority queue -> heap구조가 좋음
 - » FCFS : equal-priority
 - » SJF : $p = 1/T$ (T = 차기 CPU 버스트 시간 예측 값)
 - » priority요인(OS내부)
 - 시간 제한(time limits)
 - 메모리 요구량
 - 오픈 화일 수
 - (average I/O burst)/(average CPU burst) 비율 등
 - » priority요인(OS외부)
 - 프로세스 중요도
 - 컴퓨터 사용료 형태와 금액
 - 작업 부서
 - 정치적 요인 등
 - » preemptive 일 수도 non-preemptive 일 수도
 - » 문제점 = 무한 정지(blocking) 또는 기아 상태(starvation)
 - CPU를 영원히 기다림 : 결국 실행되거나 system crash 때 사라지거나
 - » (예) IBM 7094 at MIT 1973, 1967 job
 - 해결 -> aging : wait 시간 길어지면 priority 높여 줌

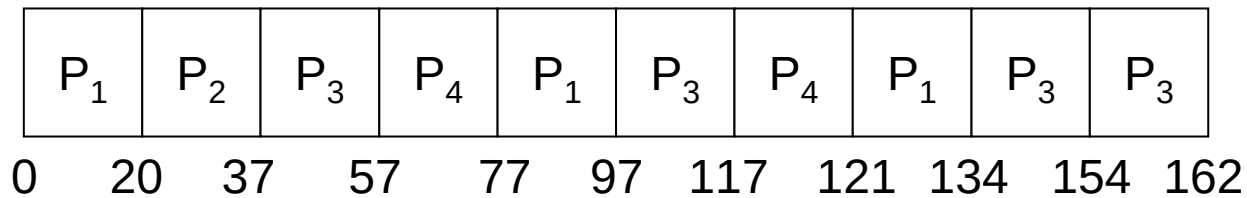
스케줄링 알고리즘(Scheduling Algorithm) ~

- 순환 할당(Round-Robin) 스케줄링
 - » FCFS + preemption (time slice 마다)
 - » ready queue = 원형 FIFO queue
 - » preemptive 임
 - » time sharing에서 time quantum의 크기가 중요
 - $1 \text{ time quantum} > \text{context switching time}$
 - $80\% \text{ CPU burst} < 1 \text{ time quantum}$
 - » p148, p150 예
- 다단계 큐(Multilevel Queue) 스케줄링
 - » 각 프로세스는 우선 순위가 다른 여러 개의 큐 중 하나에 영원히 할당:
p151 그림 5.6
 - » 각 queue는 자신의 고유한 scheduling algorithm 가짐
 - foreground (interactive) queue : RR알고리즘
 - background (batch) queue : FCFS알고리즘
 - » queue들 사이의 scheduling : 고정 우선 순위 선점 스케줄링(fixed priority preemptive scheduling)
 - » 큐 사이의 CPU time slice 할당 예
 - 80% for RR
 - 20% for FCFS
 - » 스케줄링 부담 적으나 융통성이 적음

Example: RR with Time Quantum = 20

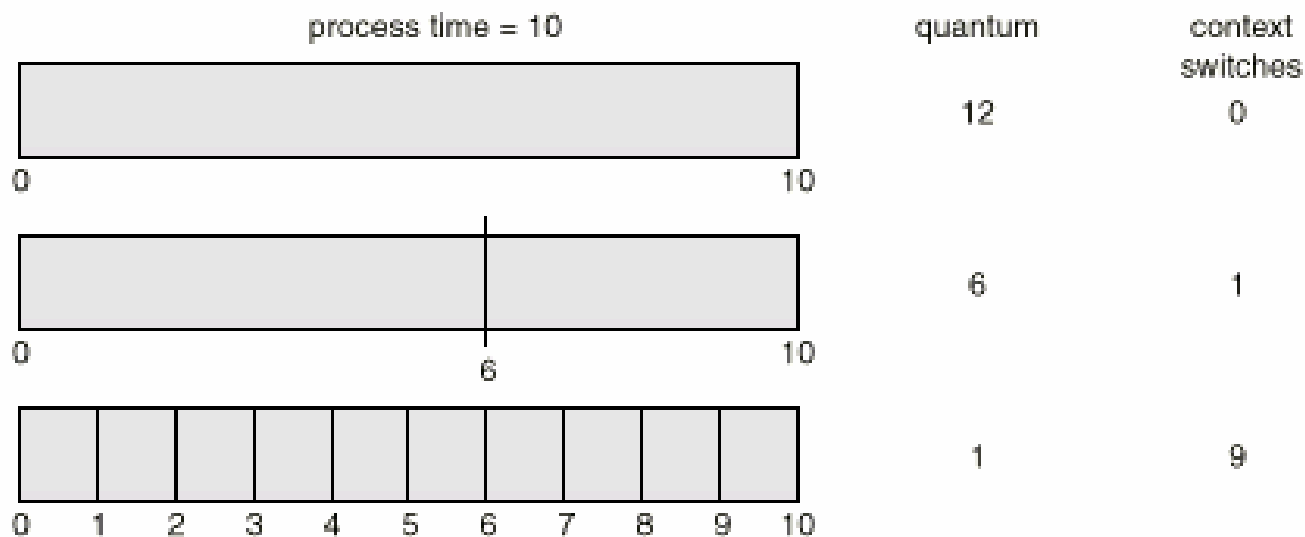
<u>Process</u>	<u>Burst Time</u>
P_1	53
P_2	17
P_3	68
P_4	24

- The Gantt chart is:

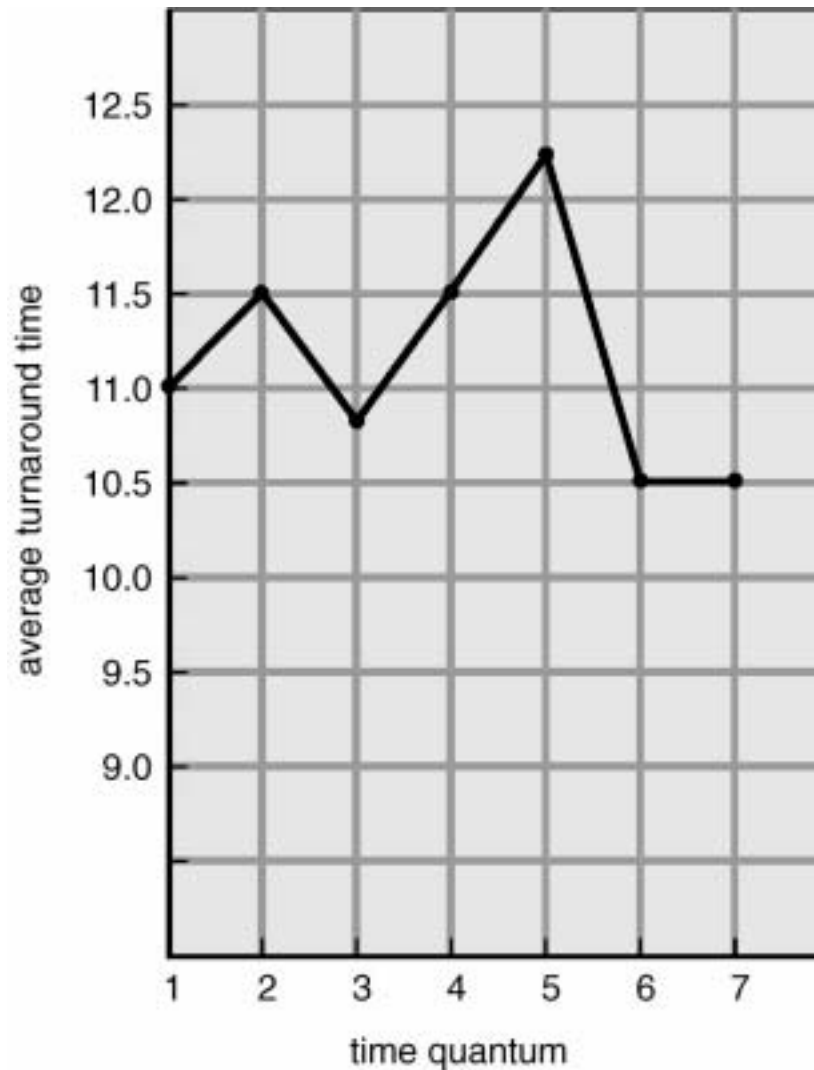


- Typically, higher average turnaround than SJF, but better *response*.

How a Smaller Time Quantum Increases Context Switches

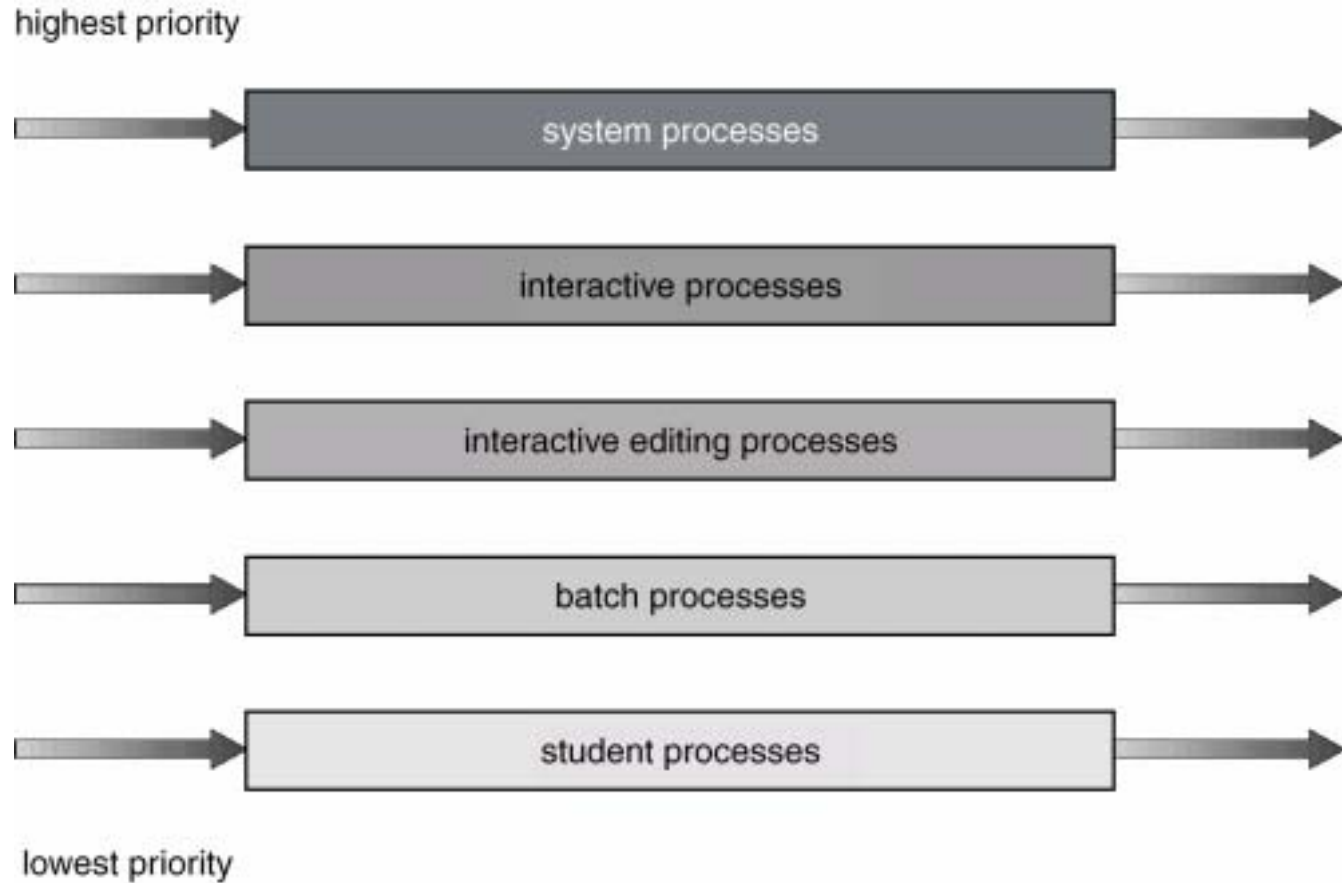


Turnaround Time Varies With The Time Quantum



process	time
P_1	6
P_2	3
P_3	1
P_4	7

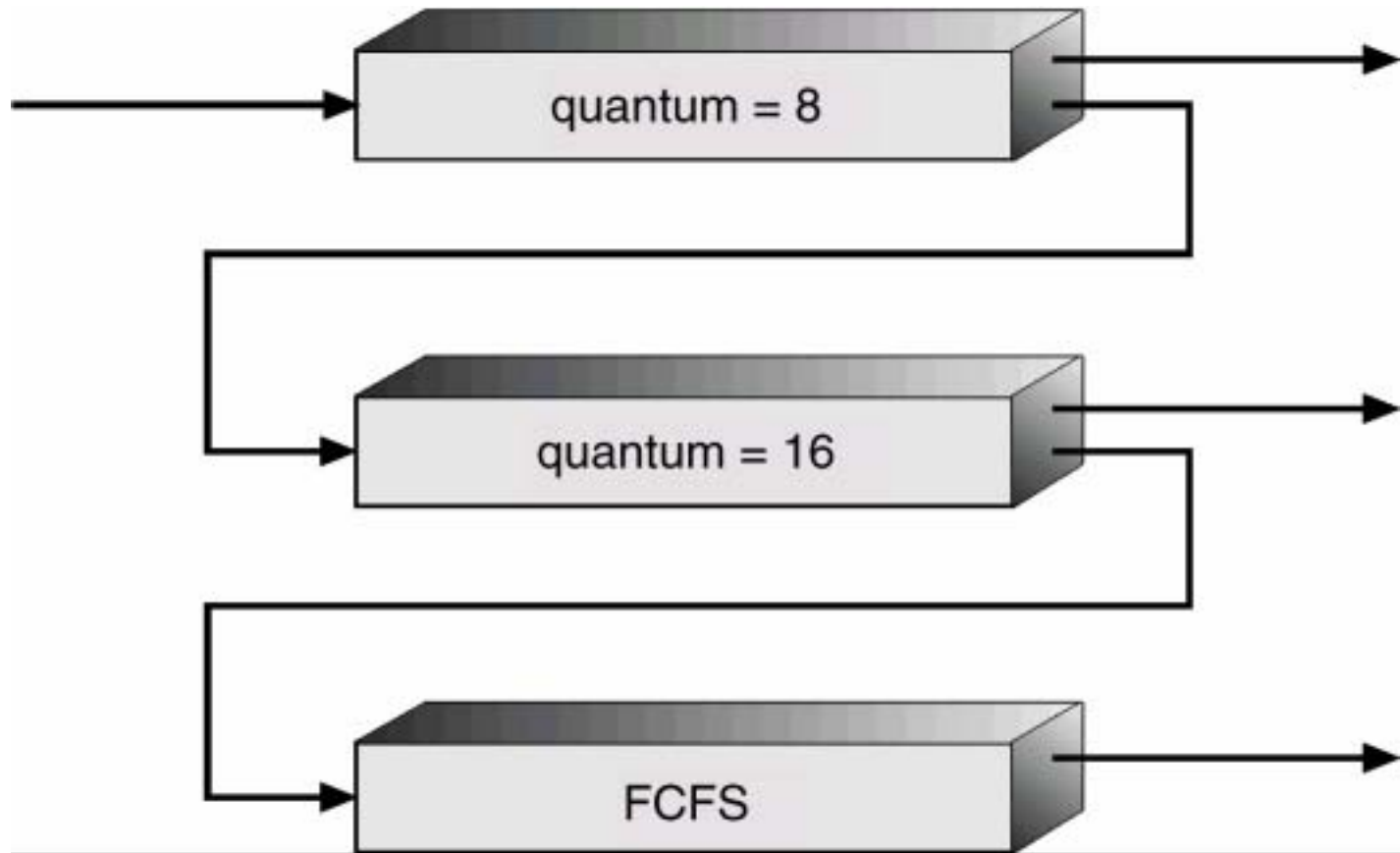
Multilevel Queue Scheduling



스케줄링 알고리즘 (Scheduling Algorithm)

- 다단계 귀환 큐(Multilevel Feedback Queue) 스케줄링
 - » 프로세스가 여러 큐 사이를 이동 : 그림 6.7
 - » 짧은 프로세스(I/O bound, interactive processes)가 우선
 - » 긴 프로세스는 자꾸 낮은 큐로 이동
 - » aging(오래 기다리면 우선순위 높여 기아 상태 예방)
 - » preemptive 임(큐 사이)
 - » the most sophisticated, the most complex
 - » 가장 일반적 -> 해당 시스템에 맞게 설정해야(configure)
 - 큐의 개수
 - 각 큐의 스케줄링 알고리즘
 - 높은 우선 순위로 올려 주는 시기
 - 낮은 우선 순위로 내려 주는 시기
 - 어느 큐에 들어갈 것인가
- HRN(Highest-Response-ratio Next) 스케줄링
 - » 1971 Brinch Hansen
 - » SJF의 단점 보완
 - » $\text{dynamic priority} = (\text{time waiting} + \text{service time}) / (\text{service time})$
 - » 오래 기다린 프로세스는 time waiting이 증가하므로 priority 커지고
 - » short process일수록 priority 커짐
 - » non-preemptive 임

Multilevel Feedback Queues



Example of Multilevel Feedback Queue

- Three queues:
 - » Q_0 – time quantum 8 milliseconds
 - » Q_1 – time quantum 16 milliseconds
 - » Q_2 – FCFS
- Scheduling
 - » FCFS queue Q_0 에 새로 들어온 작업이 8 milliseconds 동안 CPU를 받고도 작업이 끝나지 않으면 선점되어 queue Q_1 으로 이동
 - » Q_1 의 작업은 FCFS로 16 additional milliseconds을 받고 그래도 끝나지 않으면 선점되어 queue Q_2 로 이동

Unix 프로세스 스케줄링의 예

- $CPU = \text{decay}(CPU) = CPU/2$
- 우선순위 = (최근의 CPU 사용량)/2 + (기본 수준 사용자 우선순위 60)
- Unix 시스템의 최고 우선순위는 0 (우선순위 값이 작을 수록 우선순위 높음)

시간	프로세스A		프로세스B		프로세스C	
	우선순위	CPU계수	우선순위	CPU계수	우선순위	CPU계수
0	60	0 1 2 ...	60	0	60	0
1	75	60 30	60	0 1 2 ...	60	0
2	67	15	75	60 30	60	0 1 2 ...
3	63	7 8 9 ...	67	15	75	60 30
4	76	67 33	63	7 8 9 ...	67	15
5	68	16	76	67 33	63	7

다중 프로세서 스케줄링(Multiple-Processor Scheduling)

- 동종 다중 프로세서(homogeneous multiprocessor)
 - » 프로세서는 큐에 있는 어떤 프로세스(any processes)든 실행
 - » 부하 공유(load sharing)
 - 별도의 준비 큐(separate ready queue)
 - 공동 준비 큐(common ready queue)
- 이종 다중 프로세서(heterogeneous multiprocessor)
 - » 프로세서는 정해진(dedicated) 프로세스만 실행
 - » distributed systems
- 공동 준비 큐 동종 다중 프로세서 시스템(common ready queue on homogeneous multiprocessor)의 스케줄링
 - » 각 프로세서가 스스로 스케줄링(self-scheduling) : symmetric (SMP)
 - » master-slave(asymmetric) : asymmetric
 - master만 kernel system data structure에 접근 가능
 - 한 프로세서가 다른 프로세서 스케줄링
 - » master server : all system activity
 - » client : user code only

Real-Time Scheduling ~

- **hard real-time system**
 - » 보조기억장치나 가상기억 장치사용 시스템에서는 불가능
 - » 특수 H/W상에서 수행되는 특수 S/W로 구성됨
- **soft real-time system**
 - » 중요 프로세스가 우선(general-purpose computer system에서도 가능)
 - » multimedia, high-speed interactive graphics등(soft real-time computing이 필요)
- **soft real-time OS 기능의 요구사항**
 1. 우선순위 스케줄링을 해야 함(must have priority scheduling)
 - 실시간 프로세스는 최상위 우선 순위를 유지해야 함
 2. 디스패치의 지연시간이 최소여야 함 (the dispatch latency must be small)
 - 대부분의 OS: context switching 하려면 실행중인 system call이 완료되거나 I/O를 위해 block되기를 기다려야 함

-> 해결

 - ① system call을 preemptible하게
 - » 긴 system call안에 preemption points(더 높은 우선 순위의 프로세스가 있나 check, 있으면 interrupt)
 - ② kernel전체를 preemptible하게
 - » Kernel data보호를 위한 synchronization 필요.
 - » (예) Solaris 2 :
 - 우선순위 역전(priority inversion)
 - 즉 kernel data 수정 중일 때는 control을 넘겨 주지 않음

(cf.) Mach : threads one nonpreemptible, threads are synchronous

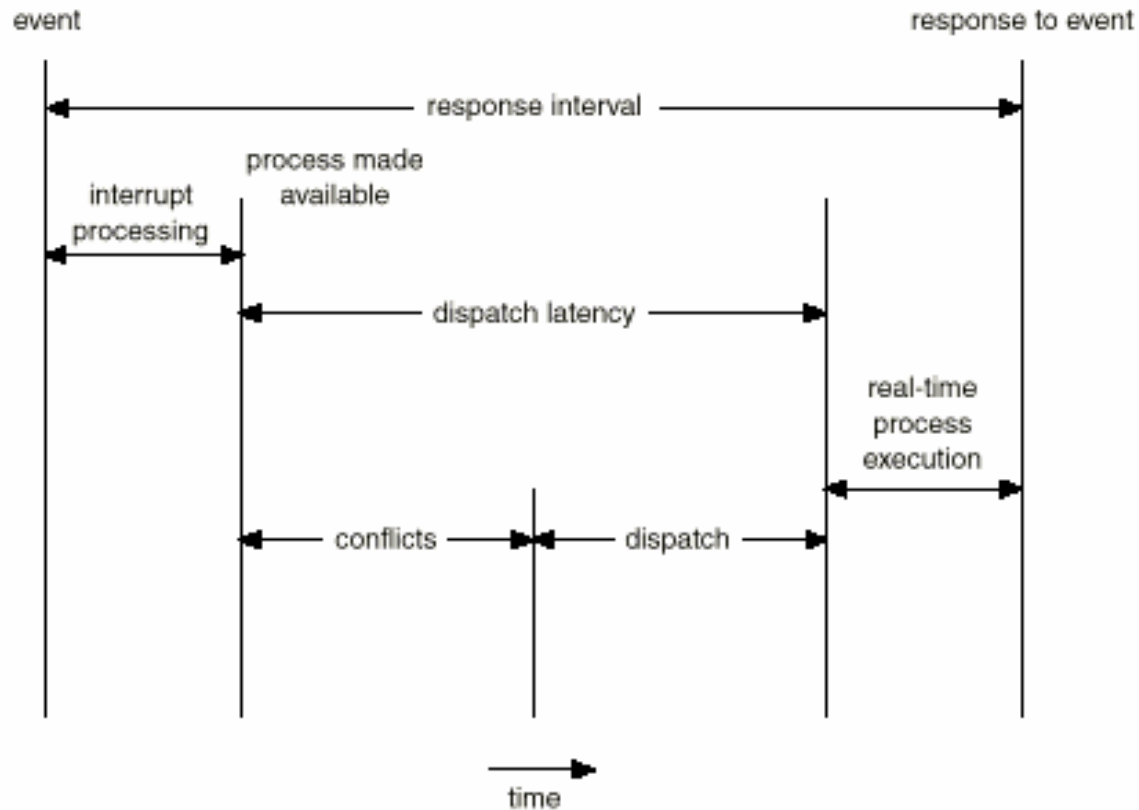
Real-Time Scheduling

- 우선순위 역전(priority inversion)
 - » kernel data 수정중인 우선순위가 낮은 프로세스가 선점하려는 우선순위 프로세스에 우선하여 수행됨
- 우선순위 상속 프로토콜(priority inheritance protocol)
 - » kernel data 수정중인 낮은 우선 순위의 프로세스가 선점하려는 프로세스의 높은 우선순위를 상속 받음, 끝나면 원래의 값으로
- 갈등 단계(complict phase)의 내용 : 그림 6.8
 1. 커널에서 실행 중인 프로세스를 선점
 2. 자원 회수(우선 순위 낮은 프로세스는 우선 순위 높은 프로세스가 요구하는 자원을 놓아줌)

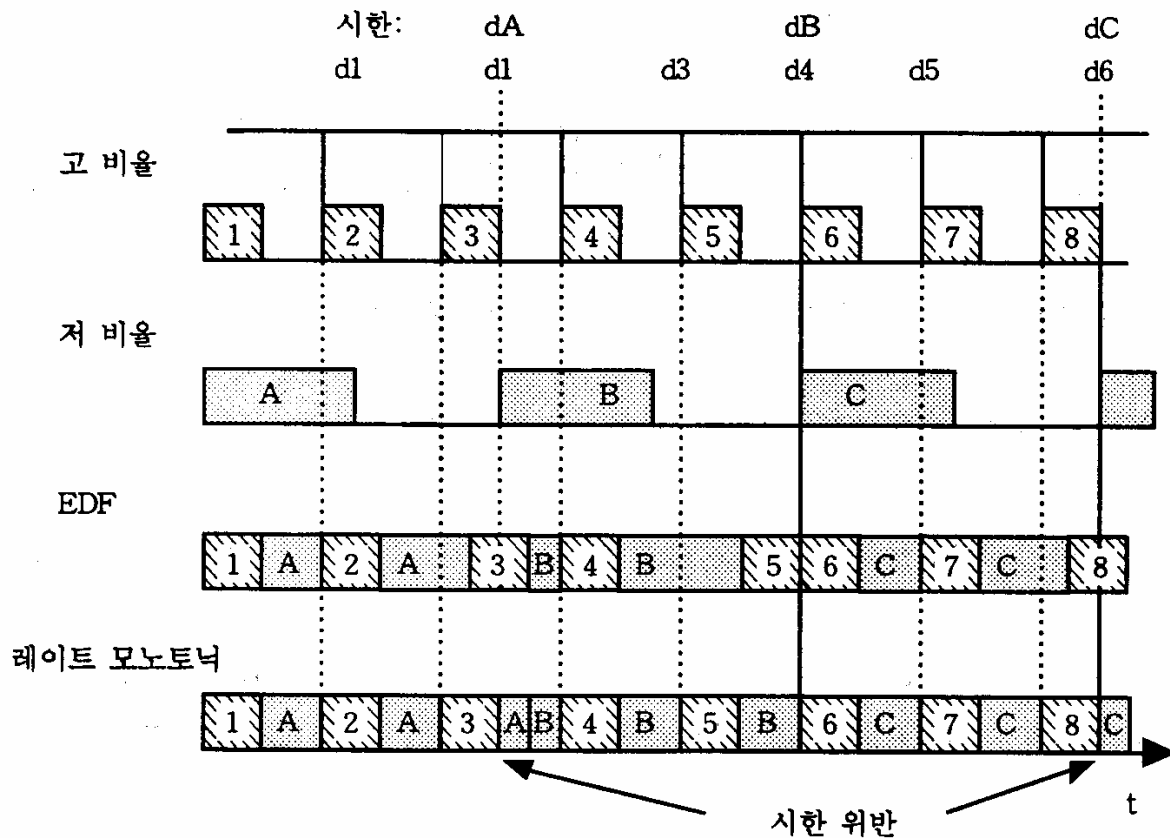
(예) Solaris 2

- dispatch latency nonpreemptible = 100 ms
- dispatch latency preemptible = 2 ms

Dispatch Latency



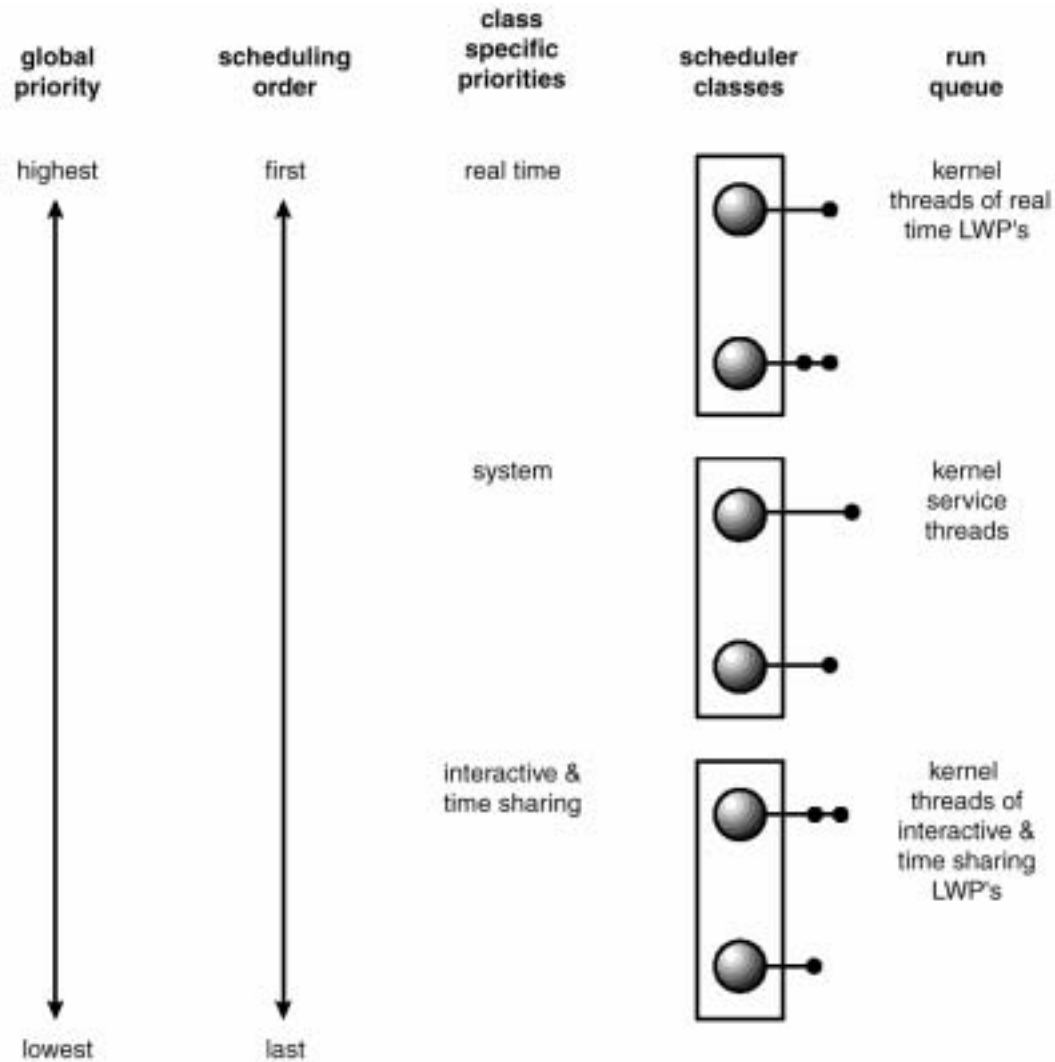
Rate Monotonic vs. EDF



Thread Scheduling

- 2 thread levels
 - » User level: process local scheduling
 - threads library가 사용 가능한 LWP에 thread를 할당하는 방법
 - » Kernel level: system global scheduling
 - kernel이 다음 실행할 kernel thread를 결정하는 방법
- Solaris 2 Scheduling
 - » priority-based process scheduling
 - » 4 classes (그림 6.9 참조)
 - real time: 최상위 우선순위
 - system: 우선순위 결정된 후 불변
 - interactive: multi-level feedback queue scheduling
 - » windowing application에 높은 우선순위
 - time sharing: default class, multi-level feedback queue scheduling
 - » the higher the priority, the smaller the time slice
 - interactive processes
 - » the lower the priority, the larger the time slice
 - CPU-bound processes
 - » Scheduler가 class-specific priorities를 global priorities로 변환
 - » 우선순위 같을 때는 round-robin
 - » 수행 중인 thread가 멈추는 경우
 - blocks
 - time slice 완료
 - 더 높은 우선순위의 thread가 선점(preempt)

Solaris 2 Scheduling



Java Thread Scheduling ~

- JVM은 스케줄링할 때
 - » Preemptive, Priority-Based Scheduling Algorithm 이용
 - » 우선순위 같으면 FIFO Queue 이용 (RR algorithm)
- Time slicing
 - » time-sliced: a thread runs until
 - time quantum
 - exit the Runnable state
 - preempted
 - » not time-sliced: a thread runs until
 - exit the Runnable state
 - preempted
 - yield() method로 공평한 수행 (time slicing 효과)
 - » cooperative multitasking

Time - Slicing

- Since the JVM Doesn't Ensure Time - Slicing(시스템 마다 다름), the yield() Method May Be Used:

```
while (true) {  
    // perform CPU-intensive task  
    . . .  
    Thread.yield();  
}
```

This Yields Control to Another Thread of Equal Priority.

Java Thread Scheduling

- Thread priority
 - » 최상위 우선순위 thread가 Runnable thread로 선택됨
 - » Thread 생성시 default priority(부모와 같음) -> setPriority()로 수정
 - Thread.MIN_PRIORITY : 1
 - Thread.MAX_PRIORITY : 10
 - Thread.NORM_PRIORITY: 5 (default priority)
 - setPriority(Thread.NORM_PRIORITY + 2);
 - Java - Based Round-robin Scheduler: thread with priority 6 (그림 6.11 참조)
 - » scheduler's queue에 addThread(): priority 2
 - » 실행 위해 선택되면: priority 4
 - » 스케줄러는 1 time quantum 동안 잠들고 CPU는 priority 4인 thread로
 - » 1 time quantum 후 잠에서 깨어난 스케줄러가 priority 4인 thread를 선점(preempt)
 - » CircularList class에 queue가 비었는지 알아내는 isEmpty() 추가하고 큐가 비었을 때는 잠들었다가 다시 queue 조사하게 하여 busy-wait 방지
 - JVM이 다음 실행할 thread를 스케줄하는 시점:
 - » 현재 수행 중이던 thread가 Runnable State를 빠져나갈 때
 - » 높은 우선순위의 thread가 Runnable State로 들어왔을 때
- * Note – the JVM Does Not Specify Whether Threads are Time-Sliced or Not.

Round-robin Scheduler

```
public class Scheduler extends Thread
{
    public Scheduler() {
        timeSlice = DEFAULT_TIME_SLICE;
        queue = new CircularList();
    }
    public Scheduler(int quantum) {
        timeSlice = quantum;
        queue = new CircularList();
    }
    public void addThread(Thread t) {
        t.setPriority(2);
        queue.addItem(t);
    }
    private void schedulerSleep() {
        try {
            thread.sleep(timeSlice);
        } catch (InterruptedException e) {
        };
    }
}
```

```
public void run() {
    Thread current;
    this.setPriority(6);
    while (true) {
        //get the next thread
        current = (Thread)queue.getNext();
        if ((current != null) && (current.isAlive()))
        {
            current.setPriority(4);
            schedulerSleep();
            current.setPriority(2);
        }
    }
    private CircularList queue;
    private int timeSlice;
    private static final int
        DEFAULT_TIME_SLICE = 1000;
}
```

TestThread

```
class TestThread extends Thread
{
    private String name;

    public TestThread(String id) {
        name = id;
        this.setPriority(Thread.NORM_PRIORITY);
    }
    public void run() {
        /*
         * The thread does something
         */
        while (true) {
            for (int i = 0; i < 500000; i++)
                ;
            System.out.println("I am thread " + name);
        }
    }
}
```

TestScheduler

```
public class TestScheduler
{
    public static void main(String args[]) {
        Thread.currentThread().setPriority(Thread.MAX_PRIORITY);
        Scheduler CPUScheduler = new Scheduler();
        CPUScheduler.start();
        TestThread t1 = new TestThread("Thread 1");
        t1.start();
        CPUScheduler.addThread(t1);
        TestThread t2 = new TestThread("Thread 2");
        t2.start();
        CPUScheduler.addThread(t2);
        TestThread t3 = new TestThread("Thread 3");
        t3.start();
        CPUScheduler.addThread(t3);
    }
}
```

알고리즘 평가(Algorithm Evaluation)

- 결정성 모형화(Deterministic Modeling)
 - » 작업부하(workload)에 따른 성능 비교 : p158-159 예제 참고
 - 반환시간
 - 대기시간 등
 - » CPU 버스트 시간 등 많은 정확한 정보 요구
- 큐잉 모형(Queueing Models)
 - » CPU-I/O 버스트 뿐 아니라 프로세스 도착시간의 분포도 평가에 고려해야
 - » 도착율과 서비스율 알면 이용율, 평균 큐 길이, 평균대기시간 알 수 있음
 - » Little의 공식 : (평균 큐 길이) = (도착 율) x (평균대기 시간)
 - » 모든 경우에 적용가능하나 근사치일 뿐
- 모의 실험(Simulations)
 - » 소프트웨어 자료구조로 clock variable, system state variables 등 표현하고 통계
 - » 사건 분포는 수학적(균일, 지수, 포아송), 또는 경험적으로 정의
 - » 사건 생성 순서 정보 제공 위해 trace tapes 이용
 - » 정확하나 비용이 많이 듦
- 구현(Implementation)
 - » 가장 정확하나 비용 많고 사용자 적응이 문제
 - » 가장 융통성 있는 스케줄링 알고리즘(flexible scheduling algorithm)
 - ➔ tunable scheduling
 - 시스템 관리자가 응용 영역에 따라 스케줄러 변수들을 변경할 수 있음
 - 몇몇 Unix 버전에서 세밀한 tuning 가능
 - Yield() 나 setPriority() API 이용하여 응용의 동작 예측 가능하게 함

Evaluation of CPU Schedulers by Simulation

